

# Visual Exploratory Data Analysis (VisEDA): PART A

## Image Exploratory Data Analysis

A Guide to Image Dataset Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

## Preface

---

The practice of Exploratory Data Analysis (EDA) has been a cornerstone of data science since its formalisation by John Tukey. For structured data, analysts can rely on mature profiling tools; image datasets require a different set of checks because spatial resolution, pixel distributions, colour, texture, compression, exposure, blur, and duplicate content all influence computer-vision models.

VisEDA was created to bring systematic dataset inspection to visual and multimodal data. ImageEDA operates at two levels: each image is analysed into an ImageRecord, and those records are aggregated to reveal dataset-level distributions, quality problems, class imbalance, duplicate groups, normalisation statistics, and visual biases.

This edition preserves the structure and core explanations of the original ImageEDA manual while updating the parts that had become incomplete or too brief. In particular, the HTML report, the current package command-line interface, the full test workflow, report contents, generated outputs, and the distinction between CLI flags and test-script flags are now documented in detail.

Isaac Osei Agyemang

August 2026

# Table of Contents

|                                                                 |    |
|-----------------------------------------------------------------|----|
| <b>Preface</b> .....                                            | 2  |
| <b>Table of Contents</b> .....                                  | 3  |
| <b>Chapter 1 — Introduction</b> .....                           | 6  |
| 1.1 What is VisEDA? .....                                       | 6  |
| 1.2 Why Image EDA Matters .....                                 | 6  |
| 1.3 Design Philosophy.....                                      | 6  |
| 1.4 Installation.....                                           | 6  |
| 1.5 Supported Data Formats .....                                | 7  |
| <b>Chapter 2 — Quick Start</b> .....                            | 8  |
| 2.1 Loading a Dataset from a Directory .....                    | 8  |
| 2.2 Getting a Summary .....                                     | 8  |
| 2.3 Generating Visualisations .....                             | 8  |
| 2.4 Getting Normalisation Statistics .....                      | 8  |
| 2.5 Generating an HTML Report .....                             | 9  |
| 2.6 Command-Line Interface.....                                 | 9  |
| 2.7 Running the Full ImageEDA Test Script .....                 | 9  |
| <b>Chapter 3 — The ImageEDA Class</b> .....                     | 10 |
| 3.1 Constructor .....                                           | 10 |
| 3.2 load() — Load from Directory or File List .....             | 10 |
| 3.3 load_arrays() — Load NumPy Arrays Directly.....             | 10 |
| 3.4 summary() — Full Statistics Dictionary.....                 | 10 |
| 3.5 normalization_stats() — Training Normalisation Values ..... | 11 |
| 3.6 report() — HTML Report .....                                | 11 |
| <b>Chapter 4 — What Is Analysed</b> .....                       | 12 |
| 4.1 Dataset Inventory .....                                     | 12 |
| Total / Valid / Corrupt Count .....                             | 12 |
| Format Distribution .....                                       | 12 |
| Colour Mode and dtype Distribution .....                        | 12 |
| 4.2 Spatial Analysis.....                                       | 12 |
| 4.3 Pixel Statistics and Dataset RGB .....                      | 12 |
| 4.4 Quality Metrics .....                                       | 12 |
| Brightness .....                                                | 12 |
| Contrast .....                                                  | 12 |
| Sharpness (Laplacian Variance) .....                            | 12 |
| Noise Estimate .....                                            | 13 |
| Overexposed and Underexposed Pixel Fraction.....                | 13 |
| JPEG Blockiness Score .....                                     | 13 |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| Pixel Entropy .....                                               | 13        |
| 4.5 Colour Analysis.....                                          | 13        |
| Colour Temperature.....                                           | 13        |
| Greyscale-Like Detection .....                                    | 13        |
| 4.6 Texture Analysis (GLCM).....                                  | 13        |
| 4.7 Frequency Analysis (FFT).....                                 | 13        |
| 4.8 Duplicate Detection .....                                     | 14        |
| Exact Duplicates (MD5).....                                       | 14        |
| Near-Duplicates (Perceptual Hash) .....                           | 14        |
| 4.9 Labels and Class Imbalance .....                              | 14        |
| <b>Chapter 5 — Visualisations .....</b>                           | <b>15</b> |
| 5.1 plot() — Main EDA Dashboard .....                             | 15        |
| Row 1: Dataset Overview and Label Distribution .....              | 16        |
| Rows 2–4: Spatial, Quality, Exposure, Colour, and Entropy .....   | 16        |
| Rows 5–6: RGB, GLCM, FFT, Cross-Metric Relationships .....        | 16        |
| 5.2 plot_colour() — Colour Analysis Dashboard .....               | 16        |
| 5.3 plot_quality() — Quality Analysis Dashboard .....             | 17        |
| 5.4 plot_texture() — Texture & Frequency Analysis .....           | 17        |
| 5.5 plot_samples() — Sample Image Grid.....                       | 18        |
| 5.6 plot_class_samples() — Per-Class Sample Grid .....            | 19        |
| 5.7 plot_average_image() — Dataset Mean and Variance Image.....   | 20        |
| 5.8 plot_channel_correlation() — R/G/B Scatter Matrix .....       | 20        |
| 5.9 plot_duplicates() — Duplicate Group Viewer .....              | 21        |
| <b>Chapter 6 — Advanced Usage.....</b>                            | <b>23</b> |
| 6.1 Working with Large Datasets .....                             | 23        |
| 6.2 Accessing Per-Image Records .....                             | 23        |
| 6.3 Using VisEDA in a Training Pipeline.....                      | 23        |
| 6.4 Comparing Two Datasets .....                                  | 23        |
| 6.5 Duplicate-Aware Dataset Splitting.....                        | 23        |
| 6.6 Exporting ImageRecord Data .....                              | 24        |
| <b>Chapter 7 — Interpreting Results and Common Findings.....</b>  | <b>25</b> |
| 7.1 Red Flags to Watch For .....                                  | 25        |
| 7.2 Cats-and-Dogs Dataset: Findings from the Attached Report..... | 25        |
| <b>Chapter 8 — API Reference .....</b>                            | <b>26</b> |
| 8.1 Return and Failure Behaviour.....                             | 26        |
| <b>Chapter 9 — HTML Report, CLI, and Test Workflow .....</b>      | <b>27</b> |
| 9.1 HTML Report Architecture.....                                 | 27        |
| 9.2 What the Report Does Not Contain .....                        | 27        |
| 9.3 Report Generation and Reproducibility .....                   | 27        |

9.4 Current viseda image CLI.....27

9.5 CLI Examples .....28

9.6 Package CLI versus testing.py.....28

9.7 Complete User-Dataset Test Command .....28

9.8 Test Coverage .....28

9.9 Files Generated by --save-plots .....29

9.10 Troubleshooting .....29

**Chapter 10 — Roadmap.....30**

    Version 1.0.0.....30

    Roadmap .....30

**Appendix A — ImageRecord Fields .....31**

**Appendix B — summary() Schema.....32**

**Appendix C — Metric and Threshold Reference.....33**

**Appendix D — Glossary.....34**

**Appendix E — Acknowledgements .....35**

# Chapter 1 — Introduction

## 1.1 What is VisEDA?

VisEDA (Visual Exploratory Data Analysis) is a Python library for comprehensive exploratory analysis of image, hyperspectral, point-cloud, video, and text datasets. ImageEDA is the image-specific component. It converts every valid image into a consistent record of spatial, photometric, quality, colour, texture, frequency, and hash-based statistics, then aggregates those records across the dataset.

The goal is not to replace visual inspection but to make it systematic. A dataset containing tens of thousands of files cannot be understood reliably by opening a few random examples. ImageEDA therefore combines numerical summaries, reproducible plots, normalisation statistics, duplicate detection, and a self-contained HTML report.

## 1.2 Why Image EDA Matters

- **Spatial inconsistency:** widely varying image dimensions and aspect ratios affect resizing, cropping, batching, and model receptive fields.
- **Class imbalance:** a dominant class can bias optimisation and inflate apparently strong aggregate accuracy.
- **Corrupt files:** unreadable images can interrupt data loaders or silently reduce the effective dataset size.
- **Blur and noise:** poor acquisition quality can reduce discriminative detail and make labels harder to learn.
- **Exposure problems:** extreme underexposure or overexposure removes useful visual information.
- **Colour bias:** systematic channel or colour-temperature differences can become spurious predictive cues.
- **Duplicate leakage:** exact or near-duplicate images can appear in both training and validation sets and produce over-optimistic evaluation results.
- **Normalisation:** dataset-specific RGB mean and standard deviation are required for principled neural-network preprocessing.
- **Texture and frequency:** datasets can differ strongly in local texture, compression artefacts, and frequency content even when labels are balanced.

## 1.3 Design Philosophy

- **Progressive disclosure.** `summary()` gives the numbers; plotting methods provide visual explanations; `report()` creates a shareable record.
- **One class per analysis session.** Construct ImageEDA once, load a dataset, then reuse the same records for summaries, plots, and reports.
- **Lazy heavy dependencies.** OpenCV, Matplotlib, and scikit-image are imported only where needed.
- **Optional expensive features.** GLCM and FFT calculations can be controlled at construction time.
- **Dataset-scale safeguards.** `max_images` limits analysis for quick previews; individual visualisations also sample files or pixels where appropriate.
- **Reproducible outputs.** Sample grids and pixel sampling use explicit random seeds in the implementation.
- **Portable reporting.** The HTML report is a single self-contained file with inline CSS and no web connection requirement.

## 1.4 Installation

```
# Core scientific/image dependencies used by ImageEDA
pip install numpy matplotlib opencv-python

# GLCM texture analysis
pip install scikit-image

# If VisEDA is installed as a package
pip install viseda

# Documentation generator in this package
pip install python-docx pillow
```

**Note:** If scikit-image is unavailable, GLCM features can be disabled with `compute_glcmm=False` in Python or `--no-glcmm` in the test script. The current package CLI does not expose a `--no-glcmm` switch.

## 1.5 Supported Data Formats

**Table 1. ImageEDA supported image inputs.**

| Format       | Extensions    | Current handling                                                                           |
|--------------|---------------|--------------------------------------------------------------------------------------------|
| JPEG         | .jpg, .jpeg   | Read through OpenCV; compression blockiness metric is especially relevant.                 |
| PNG          | .png          | Supported; alpha channels are converted to RGB for analysis.                               |
| BMP          | .bmp          | Supported through OpenCV.                                                                  |
| TIFF         | .tif, .tiff   | Supported; input is normalised to an 8-bit RGB representation for the implemented metrics. |
| WebP         | .webp         | Supported through OpenCV.                                                                  |
| GIF          | .gif          | Supported extension; analysis is image-based rather than temporal animation analysis.      |
| NumPy arrays | load_arrays() | HxWx3 or compatible arrays are converted to uint8 RGB before analysis.                     |

## Chapter 2 — Quick Start

### 2.1 Loading a Dataset from a Directory

```
# Example structure
# PetImages/
#   Cat/
#     0.jpg
#     1.jpg
#   Dog/
#     0.jpg
#     1.jpg

from viseda import ImageEDA

eda = ImageEDA(verbose=True)
eda.load("PetImages/", label_from_parent=True)
```

`label_from_parent=True` assigns the immediate parent directory name to `ImageRecord.label`. This matches common image-classification folder layouts and allows the dataset dashboard, HTML report, and label summary to compute class distributions automatically.

### 2.2 Getting a Summary

```
stats = eda.summary()

print(stats["inventory"])
print(stats["spatial"])
print(stats["pixel_stats"])
print(stats["quality"])
print(stats["colour"])
print(stats["texture"])
print(stats["frequency"])
print(stats["duplicates"])
print(stats["labels"])
```

### 2.3 Generating Visualisations

```
eda.plot()
eda.plot_colour()
eda.plot_quality()
eda.plot_texture()
eda.plot_samples(n=25, cols=5)
eda.plot_class_samples(n_per_class=6)
eda.plot_average_image(target_size=(224, 224))
eda.plot_channel_correlation()
eda.plot_duplicates(mode="exact")
eda.plot_duplicates(mode="near")
```

Every plotting method supports a `save_path` option. The dashboard-oriented methods also accept `dpi`; specialised sample/duplicate methods choose suitable default DPI internally.

### 2.4 Getting Normalisation Statistics

```
norm = eda.normalization_stats()
print(norm)

# PyTorch
import torchvision.transforms as T

transform = T.Compose([
    T.ToTensor(),
    T.Normalize(mean=norm["mean"], std=norm["std"]),
])
```



The returned channel order is R, G, B and values are scaled to [0,1]. In the current implementation, the mean is the average of per-image RGB means and the std is the average of per-image RGB standard deviations, so every image contributes equally regardless of its pixel dimensions.

## 2.5 Generating an HTML Report

```
report_path = eda.report("viseda_image_report.html")
print(report_path)
```

report() recalculates summary(), adds normalization\_stats(), and writes one standalone HTML file. The current report contains inventory badges; format and colour-mode distributions; spatial statistics; RGB dataset statistics; a copy-ready normalisation block; quality metrics; colour metrics; GLCM texture summaries; FFT frequency summaries; duplicate counts; label distribution; class imbalance; and a corrupt-file list when required.

The report uses inline CSS only. It does not depend on a JavaScript framework, external stylesheet, CDN, or network connection. This makes the file suitable for archiving with an experiment, sharing by email, or attaching to a dataset audit.

## 2.6 Command-Line Interface

```
# Current package CLI
viseda image "C:/path/to/dataset" --label-from-parent --report viseda_image_report.html --plot --save-plot
outputs_img/plot.png

# Analyse only the first 1000 resolved images
viseda image "C:/path/to/dataset" --max 1000 --plot
```

**Important CLI correction:** The current viseda image CLI supports path, --max, --label-from-parent, --report, --plot, and --save-plot. Flags such as --quick, --save-plots, --blur-thresh, and --no-glcmm belong to testing.py, not to the package CLI.

## 2.7 Running the Full ImageEDA Test Script

```
# Full run using your own dataset
python testing.py --data "C:/path/to/cats_dogs" --save-plots --report viseda_image_report.html

# Quick run: skip average-image test
python testing.py --data "C:/path/to/cats_dogs" --max 1000 --quick --save-plots

# Skip GLCM texture calculations
python testing.py --data "C:/path/to/cats_dogs" --no-glcmm --save-plots
```

The supplied testing.py is separate from the installed CLI. It validates loading, summary fields, normalisation, all major plotting functions, duplicate viewers, the HTML report, per-image records, and edge cases against a user-supplied dataset.

## Chapter 3 — The ImageEDA Class

### 3.1 Constructor

```
from viseda import ImageEDA

eda = ImageEDA(
    verbose=True,
    max_images=None,
    n_colors=8,
    phash_threshold=10,
    blur_threshold=50.0,
    compute_glcm=True,
    compute_freq=True,
)
```

**Table 2. ImageEDA constructor parameters.**

| Parameter       | Default | Purpose                                                                   |
|-----------------|---------|---------------------------------------------------------------------------|
| verbose         | True    | Print loading and progress messages.                                      |
| max_images      | None    | Maximum number of resolved image files to analyse.                        |
| n_colors        | 8       | Number of colour clusters requested by the dominant-colour visualisation. |
| phash_threshold | 10      | Maximum pHash Hamming distance used by near-duplicate grouping.           |
| blur_threshold  | 50.0    | Sharpness threshold below which ImageRecord.is_blurry becomes True.       |
| compute_glcm    | True    | Enable GLCM texture computation; requires scikit-image.                   |
| compute_freq    | True    | Enable FFT low/mid/high frequency-band analysis.                          |

### 3.2 load() — Load from Directory or File List

```
def load( self, source: Union[str, Path, List], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, ) -> "ImageEDA":
```

source can be a directory, a single supported image path, or a list of paths. Directory discovery can be recursive. If max\_images is set, the resolved list is truncated before image analysis. A labels mapping may be supplied explicitly, while label\_from\_parent=True derives labels from directory names.

### 3.3 load\_arrays() — Load NumPy Arrays Directly

```
def load_arrays( self, arrays: List[np.ndarray], labels: Optional[List[str]] = None, ) -> "ImageEDA":
```

load\_arrays() creates ImageRecord objects with identifiers such as <array\_0>. Arrays are converted through the same RGB normalisation path used by the module before statistics are calculated. File-only attributes such as file size are unavailable for array-loaded records.

### 3.4 summary() — Full Statistics Dictionary

```
def summary(self) -> Dict[str, Any]:
{
    "inventory": {...},
    "spatial": {...},
    "pixel_stats": {...},
    "quality": {...},
    "colour": {...},
    "texture": {...},
    "frequency": {...},
}
```

```
"duplicates": {...},
"labels": {...},
}
```

Most scalar distributions use a common statistical structure containing count, mean, std, min, p25, median, p75, and max. Duplicate sections return explicit path groups as well as group counts.

### 3.5 normalization\_stats() — Training Normalisation Values

```
def normalization_stats(self) -> Dict[str, List[float]]:
```

$$mean\_c = average_i(mean_{rgb}[i, c] / 255)$$

$$std\_c = average_i(std_{rgb}[i, c] / 255)$$

The function uses valid ImageRecord objects and returns three mean and three standard-deviation values. It raises RuntimeError when no valid image statistics are available.

### 3.6 report() — HTML Report

```
def report(self, output_path: str = "vised_report.html") -> str:
```

report() is intentionally thin: it verifies that the dataset has been loaded, obtains summary(), obtains normalization\_stats(), inserts the normalisation values into the report data, and passes the resulting dictionary to the internal HTML renderer. It returns the output path string after writing the file.

Because report() derives its content from the same analysed ImageRecord objects, the HTML values should match the statistics used in plots and summary() for the same ImageEDA session.

## Chapter 4 — What Is Analysed

### 4.1 Dataset Inventory

Inventory statistics establish what actually entered the analysis: total resolved files, valid images, corrupt files, corrupt paths, file-format distribution, channel/colour-mode distribution, and dtype distribution. These checks should precede model training because unreadable or unexpected files alter the effective dataset.

#### Total / Valid / Corrupt Count

total counts every resolved image record; valid excludes ImageRecord objects flagged as corrupt; corrupt\_paths lists the corresponding file paths.

#### Format Distribution

The file extension of every valid record is counted, making accidental format mixing visible.

#### Colour Mode and dtype Distribution

The summary maps channel counts to grayscale, RGB, or RGBA labels and also reports the analysed array dtype.

### 4.2 Spatial Analysis

Height, width, aspect ratio, megapixels, file size, and orientation are aggregated across all valid images.

$$\begin{aligned} \text{aspect\_ratio} &= \text{width} / \text{height} \\ \text{megapixels} &= \frac{\text{height} \times \text{width}}{1000000} \end{aligned}$$

Orientation is classified as landscape when aspect ratio > 1.05, portrait when < 0.95, and square otherwise.

### 4.3 Pixel Statistics and Dataset RGB

Each image stores per-channel mean, standard deviation, minimum, and maximum. The summary aggregates per-image mean vectors into dataset\_mean\_rgb and stores the mean of per-image RGB standard deviations as dataset\_pixel\_std\_rgb.

**Note:** These values are record-level aggregates, not a single streaming calculation over every pixel weighted by image area. If image dimensions vary greatly, remember that each image contributes equally to normalization\_stats().

### 4.4 Quality Metrics

#### Brightness

$$Y = 0.299R + 0.587G + 0.114B$$

*Image brightness is the mean of this luminance approximation.*

#### Contrast

Contrast is the standard deviation of the luminance array.

#### Sharpness (Laplacian Variance)

A grayscale image is filtered with OpenCV's Laplacian operator and the variance of the response is stored as sharpness. is\_blurry is True when sharpness < blur\_threshold.

### Noise Estimate

A 5×5 Gaussian-blurred grayscale image is subtracted from the original grayscale image. The standard deviation of this high-frequency residual is stored as `noise_estimate`.

### Overexposed and Underexposed Pixel Fraction

Pixels with luminance > 245 contribute to `overexposed_frac`; pixels with luminance < 10 contribute to `underexposed_frac`.

### JPEG Blockiness Score

The blockiness estimator averages absolute grayscale differences across horizontal and vertical 8-pixel boundaries. The metric is calculated for every analysed image, not only files with a .jpg extension.

### Pixel Entropy

A 256-bin grayscale histogram is converted to probabilities and Shannon entropy is calculated in bits.

## 4.5 Colour Analysis

RGB values are converted to HSV and CIE Lab. ImageRecord stores `mean_hsv`, `mean_lab`, mean saturation, colour temperature, and a grayscale-like flag.

### Colour Temperature

The implementation uses the ratio of mean red to mean blue: ratio > 1.1 is warm, ratio < 0.9 is cool, otherwise neutral.

### Greyscale-Like Detection

An image is flagged grayscale-like when mean HSV saturation is below 15, even though the internal analysis representation is RGB.

## 4.6 Texture Analysis (GLCM)

When `compute_glcm=True`, grayscale images are resized to 128×128 and quantised to 64 intensity levels. A symmetric, normalised GLCM is calculated at distance 1 for 0°, 45°, 90°, and 135°, and the four directions are averaged.

| Feature       | Meaning in the implementation                                                                  |
|---------------|------------------------------------------------------------------------------------------------|
| Contrast      | Weighted local intensity difference; larger values indicate stronger neighbouring differences. |
| Dissimilarity | Absolute grey-level difference weighted by GLCM probability.                                   |
| Homogeneity   | Higher when probability mass is concentrated near the GLCM diagonal.                           |
| Energy        | Square root of angular second moment; higher for more regular/repetitive texture.              |
| Correlation   | Linear dependency between neighbouring grey levels.                                            |
| ASM           | Angular second moment, the sum of squared GLCM probabilities.                                  |

## 4.7 Frequency Analysis (FFT)

When `compute_freq=True`, the grayscale image is resized to 128×128, transformed with a 2-D FFT, shifted to centre the low frequencies, and divided into radial bands. The stored values are fractions of total FFT magnitude.

| Band                   | Radial region                                 |
|------------------------|-----------------------------------------------|
| <code>freq_low</code>  | distance < 10% of the maximum radial distance |
| <code>freq_mid</code>  | $10\% \leq \text{distance} < 40\%$            |
| <code>freq_high</code> | distance $\geq 40\%$                          |

## 4.8 Duplicate Detection

### Exact Duplicates (MD5)

The MD5 digest is calculated from the analysed RGB array bytes. Valid records sharing the same digest are returned as an exact-duplicate group.

### Near-Duplicates (Perceptual Hash)

The current pHash resizes grayscale input to 32×32, applies a DCT, thresholds the top-left 8×8 coefficients against their median, and stores the resulting 64-bit integer. Pairs are grouped when the XOR Hamming distance is less than or equal to `phash_threshold`.

**Note:** The near-duplicate grouping routine compares later hashes to the first unvisited member of a group. It is a practical EDA heuristic, not a transitive clustering algorithm.

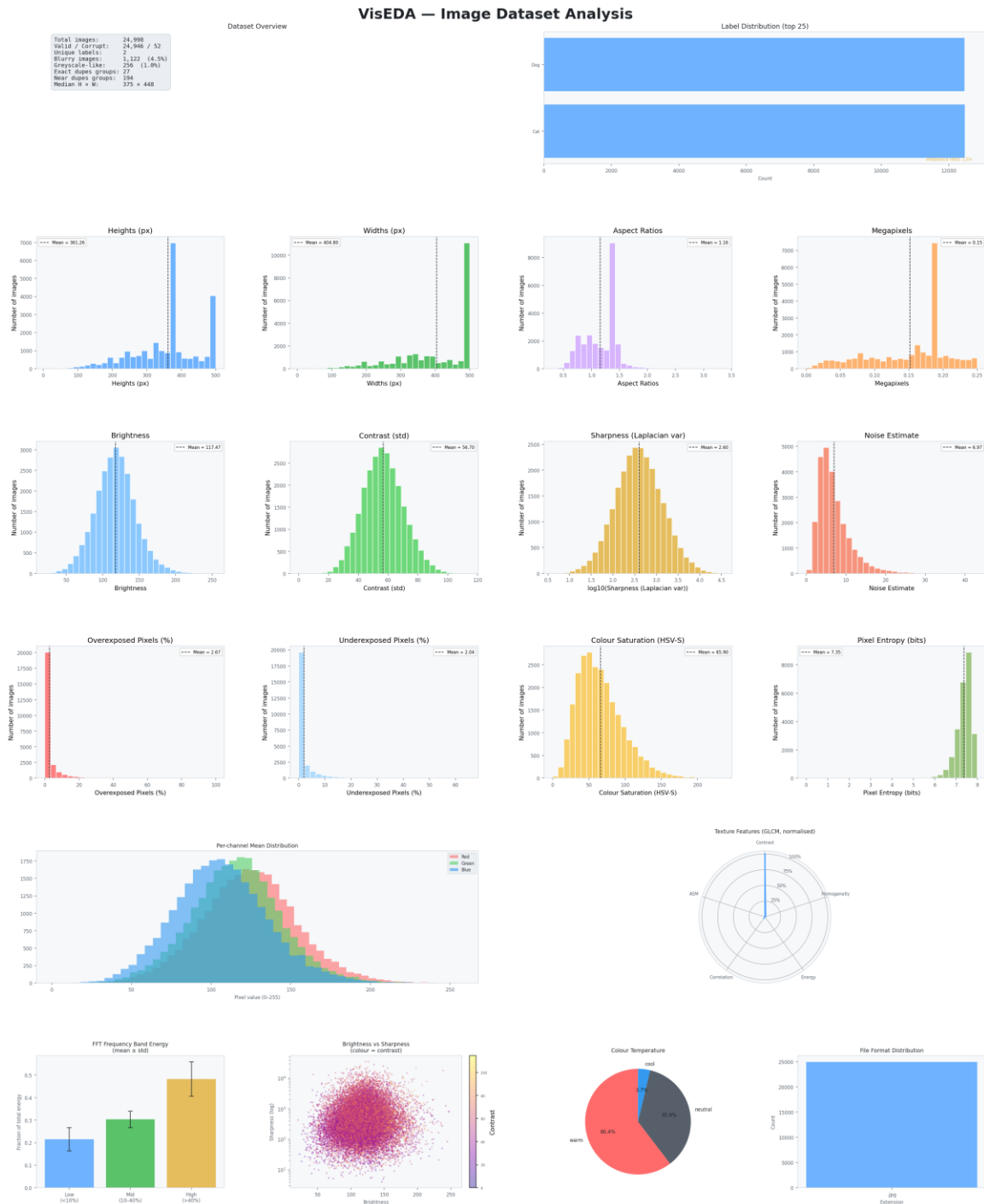
## 4.9 Labels and Class Imbalance

When labels are available, `summary()` returns `label_distribution` and `class_imbalance_ratio` = maximum class count / minimum class count.

# Chapter 5 — Visualisations

## 5.1 plot() — Main EDA Dashboard

```
def plot( self, figsize: Tuple[int, int] = (24, 28), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```



**Figure 5-1. ImageEDA Main Dataset Dashboard**

The 6×4 dashboard combines inventory, labels, spatial distributions, quality metrics, exposure, colour, entropy, RGB channel distributions, texture, FFT energy, brightness-sharpness relationships, colour temperature, and file formats.

## Row 1: Dataset Overview and Label Distribution

The overview card reports total/valid/corrupt counts, unique labels, blur, grayscale-like count, duplicate-group counts, and median dimensions. The label chart shows up to the 25 most frequent classes and annotates imbalance when multiple classes are present.

## Rows 2–4: Spatial, Quality, Exposure, Colour, and Entropy

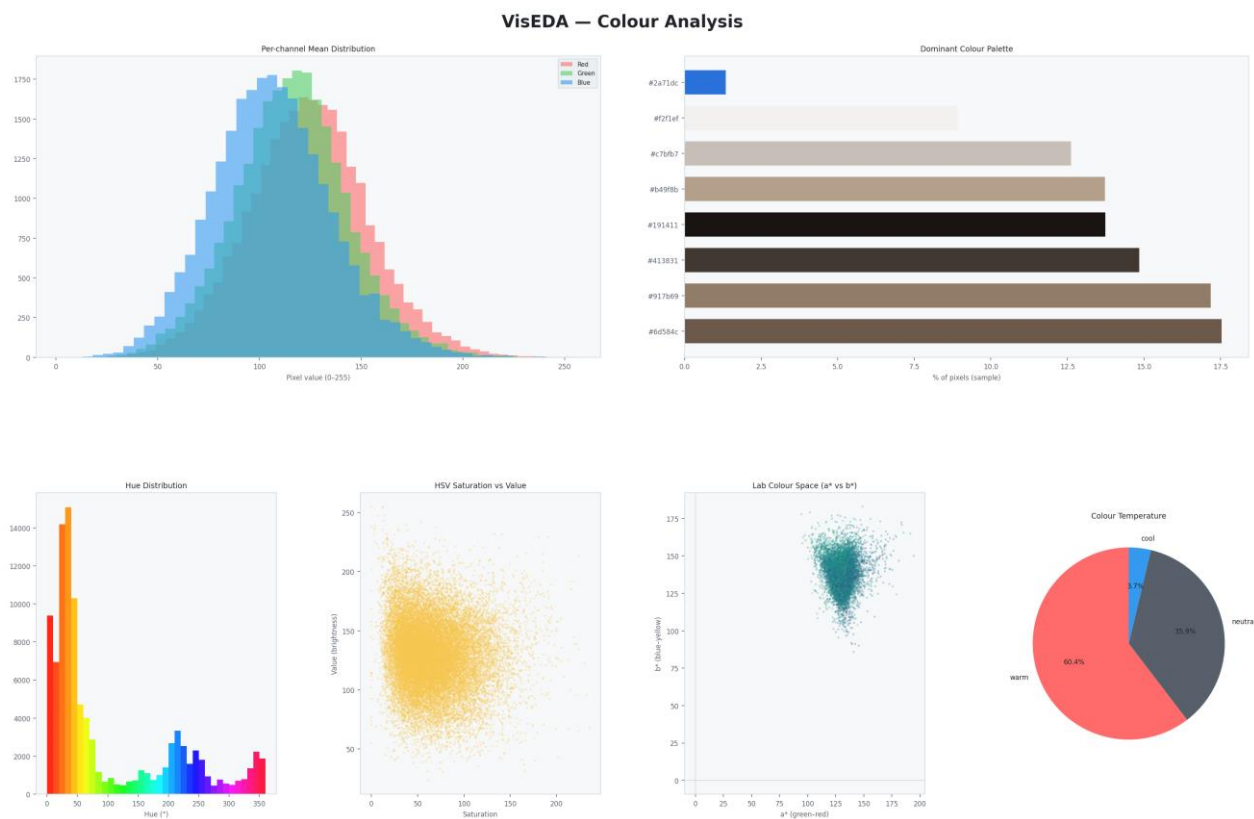
Histograms show central tendency and spread. The sharpness panel uses a logarithmic transform for display because Laplacian variance is usually strongly right-skewed.

## Rows 5–6: RGB, GLCM, FFT, Cross-Metric Relationships

The lower panels summarise channel means, GLCM features, relative FFT bands, brightness-versus-sharpness, colour temperature, and format counts.

## 5.2 plot\_colour() — Colour Analysis Dashboard

```
def plot_colour( self, figsize: Tuple[int, int] = (22, 14), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```



**Figure 5-2. ImageEDA Colour Analysis**

*Colour analysis combines per-channel mean distributions, a dominant colour palette, hue distribution, HSV saturation-versus-value, Lab  $a^*$  versus  $b^*$ , and colour-temperature composition.*



## 5.3 plot\_quality() — Quality Analysis Dashboard

```
def plot_quality( self, figsize: Tuple[int, int] = (22, 14), save_path: Optional[str] = None, dpi: int = 150, )
-> None:
```

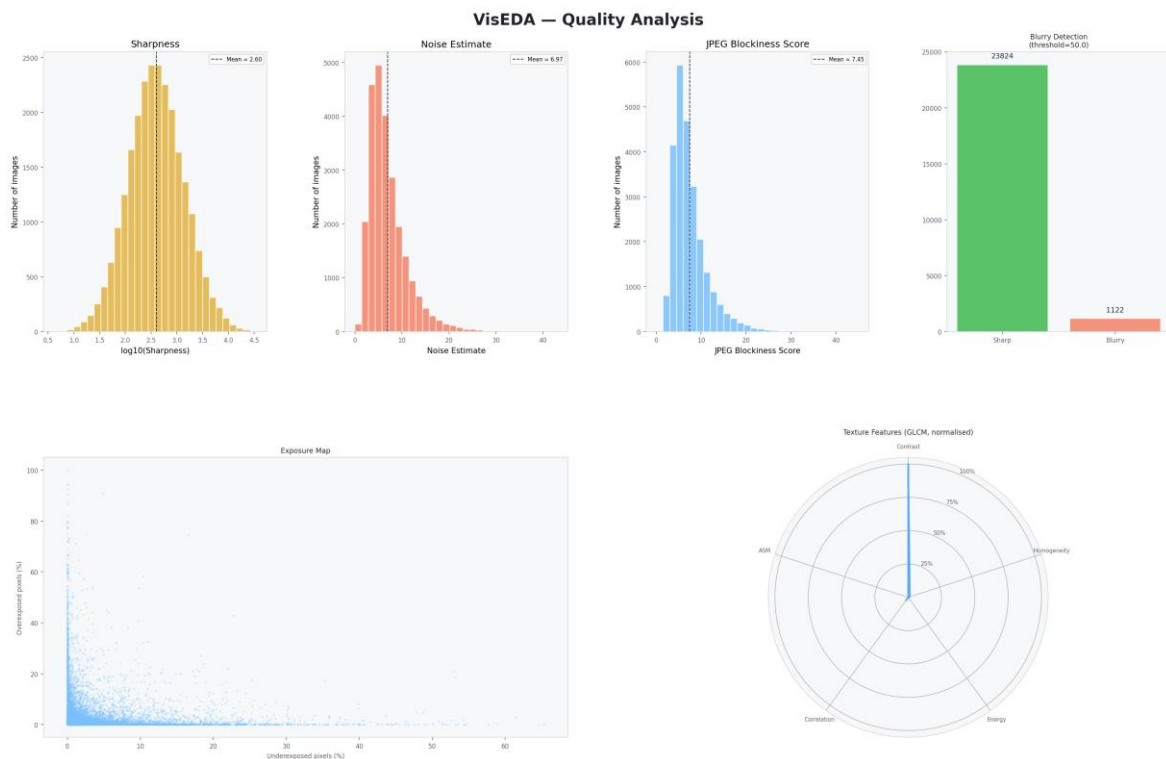


Figure 5-3. ImageEDA Quality Analysis

The quality dashboard focuses on sharpness, noise, compression score, blurry counts, the exposure map, and normalised GLCM texture features.

## 5.4 plot\_texture() — Texture & Frequency Analysis

```
def plot_texture( self, figsize: Tuple[int, int] = (22, 10), save_path: Optional[str] = None, dpi: int = 150, )
-> None:
```

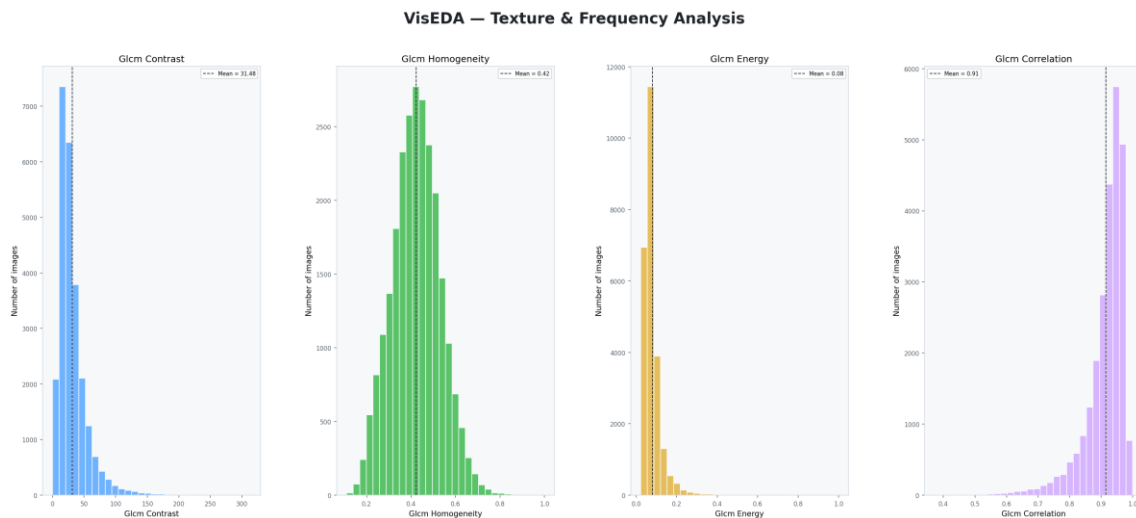
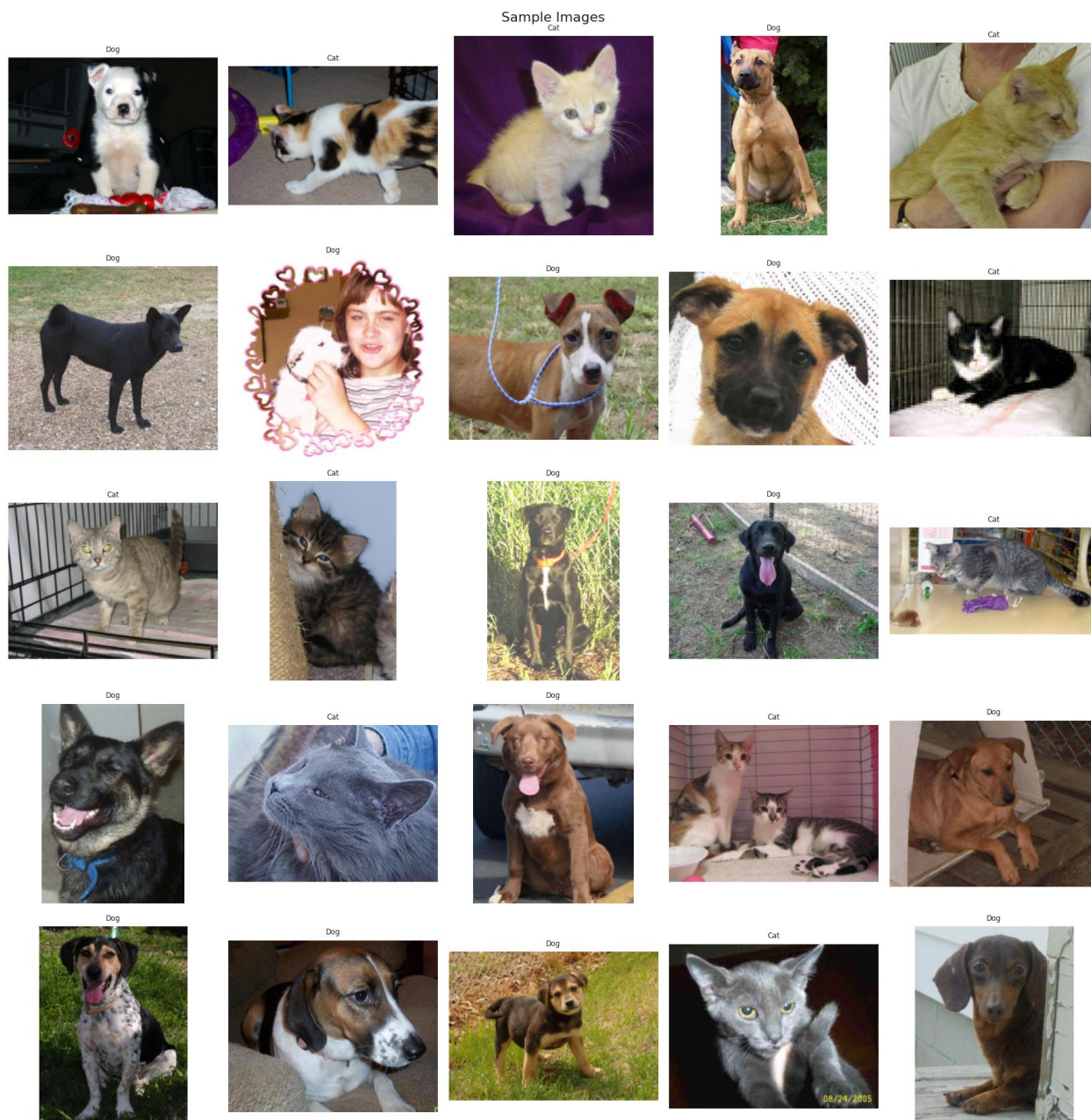


Figure 5-4. ImageEDA Texture and Frequency Analysis

The supplied texture figure shows dataset distributions for GLCM contrast, homogeneity, energy, and correlation. The main dashboard additionally summarises FFT band energy.

## 5.5 plot\_samples() — Sample Image Grid

```
def plot_samples( self, n: int = 25, cols: int = 5, label: Optional[str] = None, random_seed: int = 42, figsize: Optional[Tuple] = None, save_path: Optional[str] = None, ) -> None:
```



**Figure 5-5. Sample Images Across the Dataset**

*A reproducible random sample is useful for detecting label errors, acquisition variation, unexpected backgrounds, extreme crops, and visual artefacts that aggregate statistics cannot fully explain.*



**Figure 5-6. Sample Images — Cat**

*plot\_samples(label='Cat') restricts the random sample to one class, supporting within-class quality and diversity inspection.*



**Figure 5-7. Sample Images — Dog**

*A corresponding Dog-only sample allows qualitative comparison between class-specific composition and capture conditions.*

## 5.6 plot\_class\_samples() — Per-Class Sample Grid

```
def plot_class_samples( self, n_per_class: int = 5, save_path: Optional[str] = None, ) -> None:
```



**Figure 5-8. Per-Class Sample Images**

*This grid places representative samples from each label in a class-oriented layout, making class imbalance, label confusion, and class-specific capture bias easier to assess.*



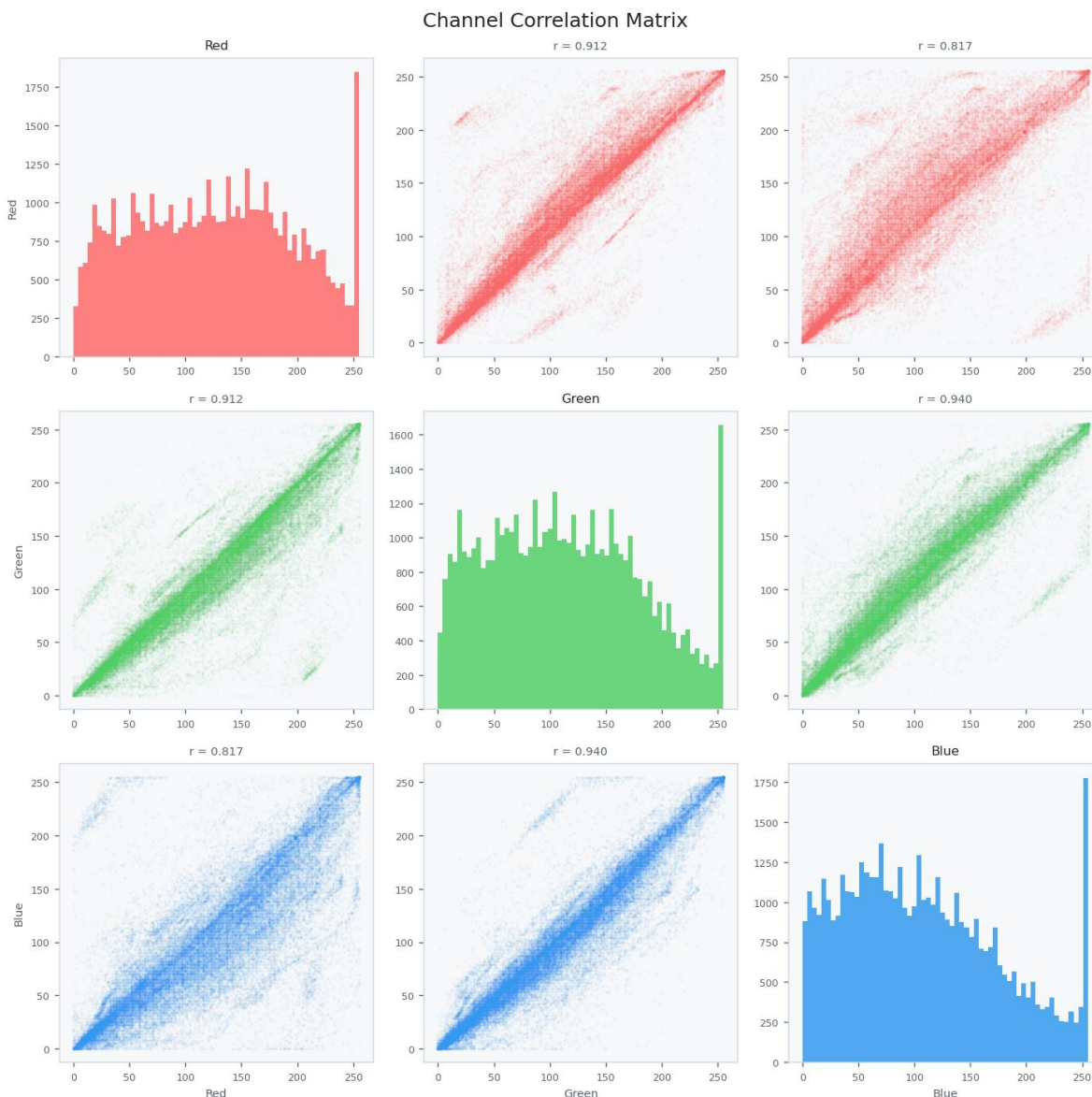
## 5.7 plot\_average\_image() — Dataset Mean and Variance Image

```
def plot_average_image( self, target_size: Tuple[int, int] = (224, 224), label: Optional[str] = None, save_path:
Optional[str] = None, ) -> None:
```

All file-backed images in the selected pool are resized to `target_size` and accumulated into a pixel-wise mean image. A second pass creates a standard-deviation image. This can reveal systematic spatial bias such as centred objects, common horizon positions, repeated backgrounds, or class-specific framing.

## 5.8 plot\_channel\_correlation() — R/G/B Scatter Matrix

```
def plot_channel_correlation( self, max_pixels: int = 50_000, save_path: Optional[str] = None, ) -> None:
```

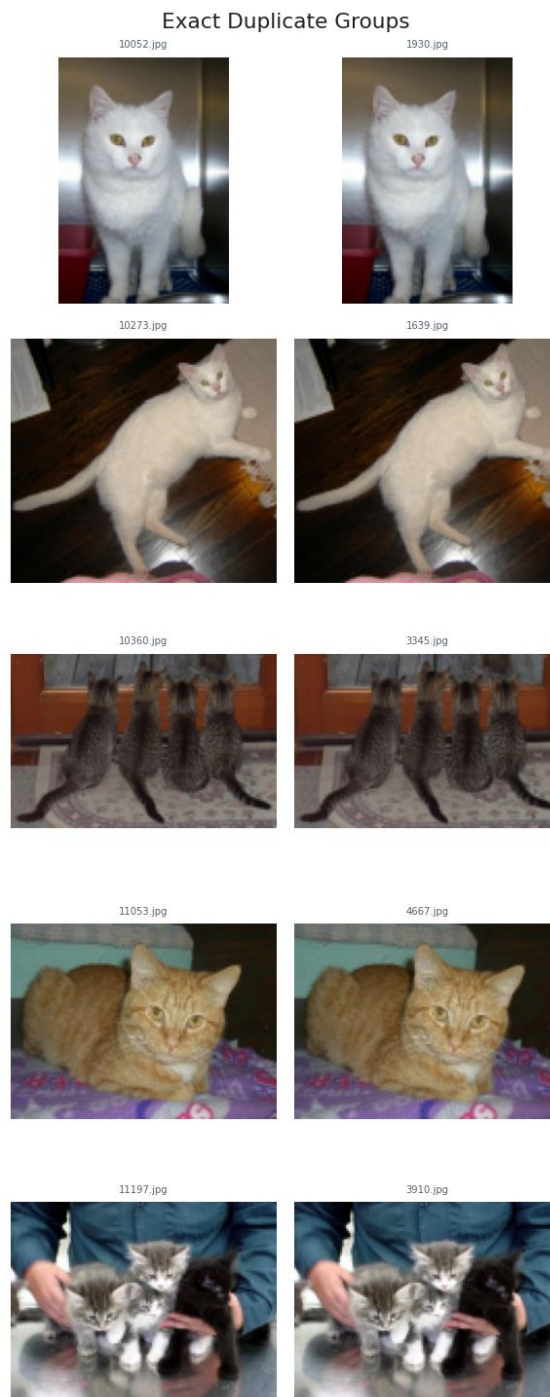


**Figure 5-9. Channel Correlation Matrix**

*Diagonal panels show channel histograms; off-diagonal panels show random pixel scatter samples and Pearson correlation coefficients. The implementation samples from at most the first 80 valid file-backed images.*

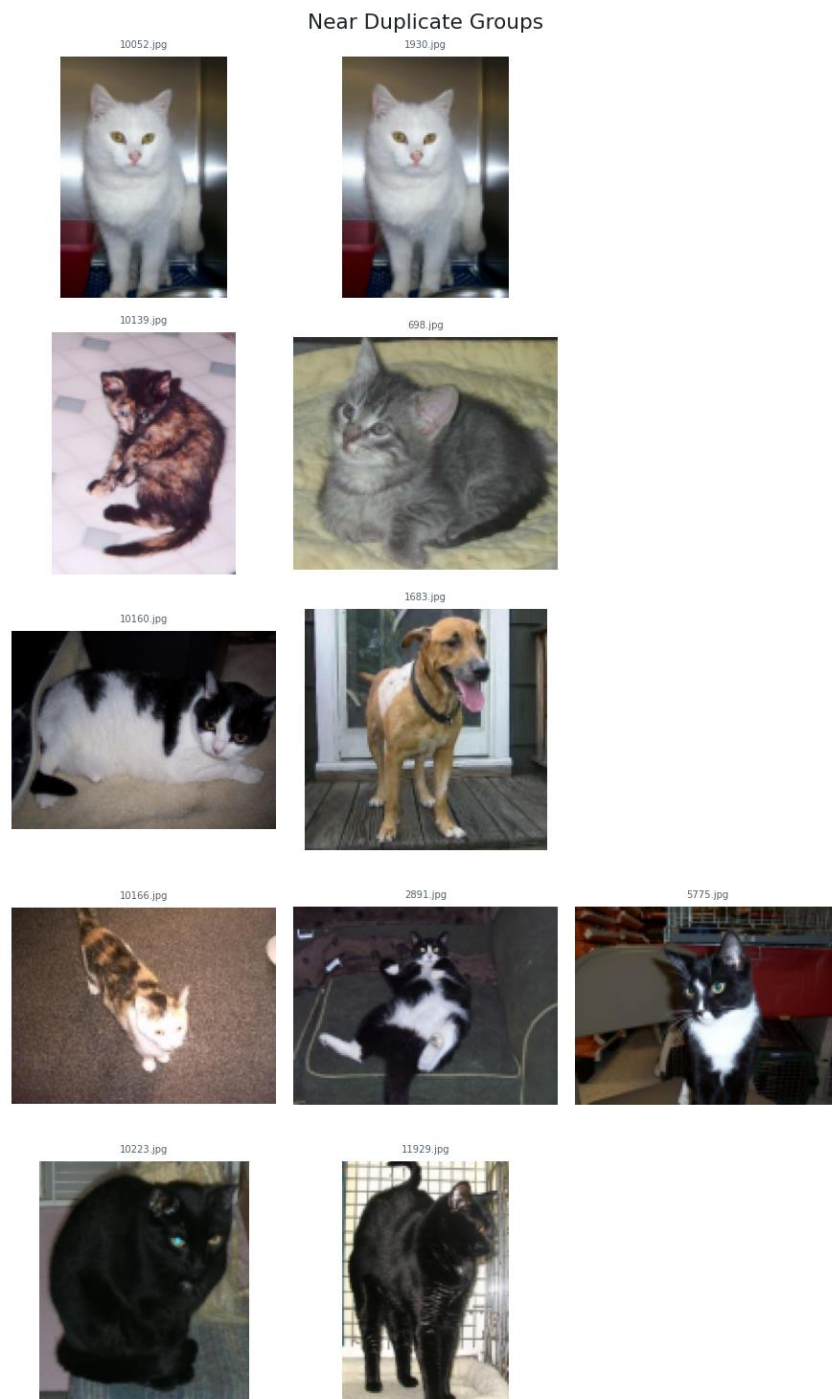
## 5.9 plot\_duplicates() — Duplicate Group Viewer

```
def plot_duplicates( self, mode: str = "near", max_groups: int = 5, save_path: Optional[str] = None, ) -> None:
```



**Figure 5-10. Exact Duplicate Groups**

*Exact groups are formed from matching MD5 hashes of the analysed RGB byte arrays and displayed as rows of images.*



**Figure 5-11. Near Duplicate Groups**

*Near-duplicate groups are based on pHash Hamming distance. They should be reviewed before creating train/validation/test splits because visually similar content can cause leakage even when filenames differ.*

## Chapter 6 — Advanced Usage

### 6.1 Working with Large Datasets

```
eda = ImageEDA(
    max_images=5000,
    compute_glm=False,
    compute_freq=True,
)
eda.load("large_dataset/", label_from_parent=True)
```

Use `max_images` for a fast first pass, then remove the limit for the final audit. GLCM is one of the more expensive per-image analyses and can be disabled when the initial question concerns inventory, resolution, exposure, or duplicates.

### 6.2 Accessing Per-Image Records

```
valid = [r for r in eda._records if not r.is_corrupt]

blurry = [r for r in valid if r.is_blurry]
darkest = sorted(valid, key=lambda r: r.brightness)[:20]
noisiest = sorted(valid, key=lambda r: r.noise_estimate, reverse=True)[:20]

for rec in blurry[:10]:
    print(rec.path, rec.sharpness, rec.label)
```

### 6.3 Using VisEDA in a Training Pipeline

```
stats = eda.summary()
norm = eda.normalization_stats()

# Example policy decisions driven by EDA
expected_size = (
    int(stats["spatial"]["height"]["median"]),
    int(stats["spatial"]["width"]["median"]),
)

exact_groups = stats["duplicates"]["exact_duplicate_groups"]
near_groups = stats["duplicates"]["near_duplicate_groups"]
```

EDA should inform, not silently perform, data cleaning. Decisions to remove blurry images, deduplicate, crop, resize, rebalance classes, or exclude corrupt files remain task-dependent.

### 6.4 Comparing Two Datasets

```
train = ImageEDA(verbose=False).load("train/", label_from_parent=True)
test = ImageEDA(verbose=False).load("test/", label_from_parent=True)

s_train = train.summary()
s_test = test.summary()

print("train brightness:", s_train["quality"]["brightness"]["mean"])
print("test brightness :", s_test["quality"]["brightness"]["mean"])
print("train RGB mean  :", s_train["pixel_stats"]["dataset_mean_rgb"])
print("test RGB mean   :", s_test["pixel_stats"]["dataset_mean_rgb"])
```

### 6.5 Duplicate-Aware Dataset Splitting

Exact and near-duplicate groups should be identified before random splitting. A practical policy is to treat every duplicate group as an indivisible unit so visually equivalent images cannot cross split boundaries.

## 6.6 Exporting ImageRecord Data

```
import pandas as pd

rows = []
for r in eda._records:
    if r.is_corrupt:
        continue
    rows.append({
        "path": r.path,
        "label": r.label,
        "height": r.height,
        "width": r.width,
        "brightness": r.brightness,
        "contrast": r.contrast,
        "sharpness": r.sharpness,
        "noise_estimate": r.noise_estimate,
        "entropy": r.entropy_val,
        "is_blurry": r.is_blurry,
    })

pd.DataFrame(rows).to_csv("image_eda_records.csv", index=False)
```



## Chapter 7 — Interpreting Results and Common Findings

### 7.1 Red Flags to Watch For

| Finding                             | Potential consequence                                 | Follow-up                                                    |
|-------------------------------------|-------------------------------------------------------|--------------------------------------------------------------|
| Corrupt files                       | Data loaders fail or silently skip samples.           | Inspect <code>corrupt_paths</code> and remove/repair files.  |
| Very wide size distribution         | Aggressive resize/crop may discard content.           | Inspect sample grids by size or class.                       |
| High blurry fraction                | Reduced fine-detail signal.                           | Review blur threshold and sample flagged files.              |
| High exposure fractions             | Clipped visual information.                           | Inspect quality dashboard and acquisition source.            |
| Strong colour-temperature imbalance | Possible dataset/source bias.                         | Compare distribution by class or split.                      |
| Low entropy cluster                 | Blank, flat, or highly uniform images may be present. | Review low-entropy records.                                  |
| Large duplicate groups              | Leakage and over-counting.                            | Deduplicate or group before splitting.                       |
| High class imbalance ratio          | Biased training and misleading accuracy.              | Use stratification, weighting, sampling, or data collection. |

### 7.2 Cats-and-Dogs Dataset: Findings from the Attached Report

The supplied HTML report represents a large cats-and-dogs analysis and provides a concrete example of the report's purpose. It records 24,998 resolved images, 24,946 valid files, and 52 corrupt files. It also reports 27 exact-duplicate groups and 194 near-duplicate groups, demonstrating why duplicate analysis should occur before final dataset splitting.

The report records a mean image height of approximately 361 px and width of approximately 405 px, with a mean aspect ratio of about 1.16. These values indicate a heterogeneous dataset rather than a fixed-resolution corpus and support the need for an explicit resizing/cropping policy.

The current report also exposes copy-ready normalisation values. For the attached analysis, the displayed means are approximately [0.4882, 0.4551, 0.4169] and standard deviations [0.2293, 0.2247, 0.2250]. These numbers are generated by the current `normalization_stats()` implementation and should be regenerated if the dataset changes.

**Note:** Example values in this section come from `vised_image_report.html` and are illustrative of the current dataset, not universal ImageEDA thresholds.

## Chapter 8 — API Reference

| Method                                | Current signature                                                                                                                                                                                    | Purpose                             |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <code>__init__</code>                 | <code>ImageEDA(verbose=True, max_images=None, n_colors=8, phash_threshold=10, blur_threshold=50.0, compute_glm=True, compute_freq=True)</code>                                                       | Create an analysis session.         |
| <code>load</code>                     | <code>def load( self, source: Union[str, Path, List], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, ) -&gt; "ImageEDA":</code>                   | Load file(s) or a directory.        |
| <code>load_arrays</code>              | <code>def load_arrays( self, arrays: List[np.ndarray], labels: Optional[List[str]] = None, ) -&gt; "ImageEDA":</code>                                                                                | Analyse in-memory arrays.           |
| <code>summary</code>                  | <code>def summary(self) -&gt; Dict[str, Any]:</code>                                                                                                                                                 | Return nested dataset statistics.   |
| <code>normalization_stats</code>      | <code>def normalization_stats(self) -&gt; Dict[str, List[float]]:</code>                                                                                                                             | Return R/G/B mean and std in [0,1]. |
| <code>plot</code>                     | <code>def plot( self, figsize: Tuple[int, int] = (24, 28), save_path: Optional[str] = None, dpi: int = 150, ) -&gt; None:</code>                                                                     | Full dataset dashboard.             |
| <code>plot_colour</code>              | <code>def plot_colour( self, figsize: Tuple[int, int] = (22, 14), save_path: Optional[str] = None, dpi: int = 150, ) -&gt; None:</code>                                                              | Colour dashboard.                   |
| <code>plot_quality</code>             | <code>def plot_quality( self, figsize: Tuple[int, int] = (22, 14), save_path: Optional[str] = None, dpi: int = 150, ) -&gt; None:</code>                                                             | Quality dashboard.                  |
| <code>plot_texture</code>             | <code>def plot_texture( self, figsize: Tuple[int, int] = (22, 10), save_path: Optional[str] = None, dpi: int = 150, ) -&gt; None:</code>                                                             | Texture/frequency view.             |
| <code>plot_samples</code>             | <code>def plot_samples( self, n: int = 25, cols: int = 5, label: Optional[str] = None, random_seed: int = 42, figsize: Optional[Tuple] = None, save_path: Optional[str] = None, ) -&gt; None:</code> | Random sample grid.                 |
| <code>plot_class_samples</code>       | <code>def plot_class_samples( self, n_per_class: int = 5, save_path: Optional[str] = None, ) -&gt; None:</code>                                                                                      | Per-class samples.                  |
| <code>plot_average_image</code>       | <code>def plot_average_image( self, target_size: Tuple[int, int] = (224, 224), label: Optional[str] = None, save_path: Optional[str] = None, ) -&gt; None:</code>                                    | Average and std-dev images.         |
| <code>plot_channel_correlation</code> | <code>def plot_channel_correlation( self, max_pixels: int = 50_000, save_path: Optional[str] = None, ) -&gt; None:</code>                                                                            | RGB correlation matrix.             |
| <code>plot_duplicates</code>          | <code>def plot_duplicates( self, mode: str = "near", max_groups: int = 5, save_path: Optional[str] = None, ) -&gt; None:</code>                                                                      | Duplicate viewer.                   |
| <code>report</code>                   | <code>def report(self, output_path: str = "vised_report.html") -&gt; str:</code>                                                                                                                     | Self-contained HTML report.         |

### 8.1 Return and Failure Behaviour

`load()` and `load_arrays()` return `self`. `summary()` and `normalization_stats()` return dictionaries. Plot methods return `None`. `report()` returns the path string. Public analysis methods require a loaded dataset and call the internal loaded-state check before operating.

## Chapter 9 — HTML Report, CLI, and Test Workflow

### 9.1 HTML Report Architecture

`ImageEDA.report()` writes one HTML document with embedded CSS. Report data come directly from `summary()` plus `normalization_stats()`. The renderer uses responsive cards and bar rows; it does not execute analysis independently.

**Table 3. Current self-contained HTML report sections.**

| Report section      | Information included                                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------------------------|
| Header badges       | Total images, valid images, corrupt count when non-zero, exact duplicate group count, near-duplicate group count. |
| Inventory           | Counts, format distribution, colour-mode distribution.                                                            |
| Spatial             | Height, width, aspect ratio, megapixels, file size, orientation distribution.                                     |
| Pixel Statistics    | Dataset mean RGB and dataset pixel std RGB.                                                                       |
| Normalisation Stats | Copy-ready <code>transforms.Normalize(mean=..., std=...)</code> block.                                            |
| Quality Metrics     | Brightness, contrast, sharpness, noise, entropy, compression, exposure, blur summary.                             |
| Colour              | Saturation, colour-temperature distribution, grayscale-like summary.                                              |
| Texture             | Every available GLCM summary returned by <code>summary()</code> .                                                 |
| Frequency           | Low, mid, and high FFT band summaries.                                                                            |
| Duplicates          | Exact and near duplicate-group counts.                                                                            |
| Labels              | Up to 25 labels in the report bar chart plus imbalance ratio.                                                     |
| Corrupt Files       | Up to the first 50 corrupt paths when corrupt files exist.                                                        |

### 9.2 What the Report Does Not Contain

The HTML report is a numerical/card report. It does not embed the Matplotlib dashboard PNGs, individual image thumbnails, duplicate-group images, or interactive JavaScript charts. Those visual assets are generated separately by the plotting methods and test script.

### 9.3 Report Generation and Reproducibility

```
from viseda import ImageEDA

eda = ImageEDA(
    max_images=None,
    blur_threshold=50.0,
    compute_glcml=True,
    compute_freq=True,
)
eda.load("PetImages/", label_from_parent=True)
eda.report("viseda_image_report.html")
```

Record the constructor settings used to generate a report. Changing `max_images`, `blur_threshold`, `phash_threshold`, `compute_glcml`, or `compute_freq` can change the resulting statistics and duplicate/quality counts.

### 9.4 Current viseda image CLI

```
viseda image PATH [--max N] [--label-from-parent]
                [--report FILE]
                [--plot]
                [--save-plot FILE]
```

| CLI argument | Meaning                                         |
|--------------|-------------------------------------------------|
| PATH         | Required image file or dataset directory.       |
| --max N      | Pass N to <code>ImageEDA(max_images=N)</code> . |

| CLI argument        | Meaning                                                             |
|---------------------|---------------------------------------------------------------------|
| --label-from-parent | Infer class labels from parent folders.                             |
| --report FILE       | Call <code>eda.report(FILE)</code> after printing the JSON summary. |
| --plot              | Generate the main <code>eda.plot()</code> dashboard.                |
| --save-plot FILE    | Pass FILE to <code>eda.plot(save_path=FILE)</code> .                |

**Note:** If `--plot` is supplied without `--save-plot`, `ImageEDA.plot()` is called with `save_path=None` and therefore follows Matplotlib's interactive display path.

## 9.5 CLI Examples

```
# Summary only
viseda image "C:/data/PetImages" --label-from-parent

# Summary + HTML report
viseda image "C:/data/PetImages" --label-from-parent --report outputs_img/viseda_image_report.html

# Summary + saved dashboard + report
viseda image "C:/data/PetImages" --label-from-parent --report outputs_img/viseda_image_report.html --plot
--save-plot outputs_img/plot.png

# Quick subset through the package CLI
viseda image "C:/data/PetImages" --max 1000 --plot
```

## 9.6 Package CLI versus testing.py

**Table 4. Do not mix the package CLI switches with the test-script switches.**

| Capability               | viseda image         | testing.py                                                                  |
|--------------------------|----------------------|-----------------------------------------------------------------------------|
| Dataset path             | Positional PATH      | --data PATH (required)                                                      |
| Limit images             | --max                | --max                                                                       |
| Parent labels            | --label-from-parent  | Always uses <code>label_from_parent=True</code> in <code>test_load()</code> |
| HTML report              | --report             | --report                                                                    |
| Main plot                | --plot / --save-plot | Generated as one of the test stages                                         |
| Save all supported plots | Not a single switch  | --save-plots                                                                |
| Quick mode               | Not exposed          | --quick skips <code>average_image</code>                                    |
| Blur threshold           | Not exposed          | --blur-thresh                                                               |
| Disable GLCM             | Not exposed          | --no-glcm                                                                   |
| Per-record validation    | No                   | Yes                                                                         |
| Edge-case tests          | No                   | Yes                                                                         |

## 9.7 Complete User-Dataset Test Command

```
python testing.py --data "C:/Users/Ike/Desktop/PetImages" --save-plots --report viseda_image_report.html
```

The test script creates an `ImageEDA` instance using `n_colors=8`, `phash_threshold=10`, `blur_threshold` from `--blur-thresh`, `compute_glcm=not --no-glcm`, and `compute_freq=True`.

## 9.8 Test Coverage

1. `load` — directory discovery, valid/corrupt counts, class detection
2. `summary` — inventory, spatial, quality, colour, pixel, texture, frequency, duplicate and label checks
3. `normalization_stats` — structure and `[0,1]` range validation
4. `main_dashboard` — `plot()`
5. `colour_dashboard` — `plot_colour()`
6. `quality_dashboard` — `plot_quality()`

7. `texture_dashboard` — `plot_texture()`
8. `sample_grid` — overall and per-class `plot_samples()`
9. `class_samples` — `plot_class_samples()`
10. `average_image` — full dataset and per-class `plot_average_image()`
11. `channel_correlation` — `plot_channel_correlation()`
12. `duplicates` — exact and near `plot_duplicates()` when groups exist
13. `html_report` — report file existence and size
14. `per_image_records` — field population/type checks
15. `edge_cases` — defensive and error-path behaviour

## 9.9 Files Generated by `--save-plots`

```

outputs_img/
├── plot.png
├── plot_colour.png
├── plot_quality.png
├── plot_texture.png
├── plot_samples.png
├── plot_samples_Cat.png      # class names depend on dataset
├── plot_samples_Dog.png
├── plot_class_samples.png
├── plot_average_image.png
├── plot_average_image_Cat.png
├── plot_average_image_Dog.png
├── plot_channel_correlation.png
├── plot_duplicates.png      # only when exact groups exist
├── plot_duplicates_near.png # only when near groups exist
└── viseda_image_report.html # default test report path

```

Some files are conditional. Duplicate viewers are skipped when no corresponding groups exist, and class-specific outputs depend on the labels found in the user dataset.

## 9.10 Troubleshooting

| Symptom                                                      | Likely cause                                                             | Action                                                                         |
|--------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| No images found                                              | Wrong dataset path or unsupported extensions.                            | Check <code>--data/PATH</code> and <code>SUPPORTED_EXTS</code> .               |
| Only one class found                                         | <code>max_images</code> truncation captured one folder first.            | Increase/remove <code>--max</code> or inspect path ordering.                   |
| Texture section empty                                        | scikit-image missing, <code>compute_glcmm=False</code> , or GLCM failed. | Install scikit-image and rerun with GLCM enabled.                              |
| Report lacks texture/frequency                               | Corresponding summary sections were empty.                               | Check constructor flags and dependencies.                                      |
| No duplicate image output                                    | No groups detected for the selected mode.                                | This is normal; inspect summary duplicate counts.                              |
| CLI rejects <code>--quick</code> or <code>--no-glcmm</code>  | Those are <code>testing.py</code> flags.                                 | Use them with <code>python testing.py</code> , not <code>viseda image</code> . |
| <code>PermissionError</code> saving <code>DOCX/report</code> | Output file is open in another application.                              | Close the file or use a different output path.                                 |

# Chapter 10 — Roadmap

---

## Version 1.0.0

- ImageEDA per-image and dataset-level statistics.
- Spatial, pixel, quality, colour, GLCM texture, FFT, duplicate, and label analysis.
- Normalisation statistics for model preprocessing.
- Main, colour, quality, texture, sample, class-sample, average-image, channel-correlation, and duplicate visualisations.
- Self-contained HTML report with normalisation snippet.
- Package CLI command: `vised image`.
- Comprehensive user-dataset test script with saved plot and report outputs.

## Roadmap

- Expose additional ImageEDA constructor controls through the package CLI.
- Optional class-conditional numerical summaries and report cards.
- More scalable near-duplicate search for very large datasets.
- Optional embedding-based similarity and outlier analysis.
- Interactive HTML plots while retaining the current portable static report.
- Explicit bit-depth and colour-profile metadata handling.
- Streaming/global pixel statistics weighted by pixel count as an alternative to equal-image weighting.
- Additional augmentation-readiness checks such as border/crop and alpha-channel diagnostics.

## Appendix A — ImageRecord Fields

| Field                 | Group     | Meaning                                        |
|-----------------------|-----------|------------------------------------------------|
| path                  | Identity  | File path or <array_N>.                        |
| label                 | Identity  | Optional class label.                          |
| file_ext              | Identity  | Lowercase extension or 'array'.                |
| file_size_kb          | Identity  | File size in KB for file-backed images.        |
| height / width        | Spatial   | Pixel dimensions.                              |
| channels              | Spatial   | Analysed channel count.                        |
| dtype                 | Spatial   | Analysed NumPy dtype string.                   |
| aspect_ratio          | Spatial   | width / height.                                |
| megapixels            | Spatial   | height × width / 1,000,000.                    |
| mean_rgb / std_rgb    | Pixel     | Per-channel mean and standard deviation.       |
| min_rgb / max_rgb     | Pixel     | Per-channel extrema.                           |
| brightness            | Quality   | Mean luminance.                                |
| contrast              | Quality   | Luminance standard deviation.                  |
| sharpness             | Quality   | Laplacian variance.                            |
| noise_estimate        | Quality   | Std of high-frequency residual.                |
| overexposed_frac      | Quality   | Fraction of luminance > 245.                   |
| underexposed_frac     | Quality   | Fraction of luminance < 10.                    |
| is_blurry             | Quality   | sharpness < blur_threshold.                    |
| compression_score     | Quality   | 8-pixel boundary blockiness estimate.          |
| entropy_val           | Quality   | Shannon entropy of grayscale histogram.        |
| mean_hsv              | Colour    | Mean H/S/V vector.                             |
| mean_lab              | Colour    | Mean OpenCV Lab vector.                        |
| saturation_mean       | Colour    | Mean HSV saturation.                           |
| color_temp            | Colour    | warm / neutral / cool.                         |
| is_grayscale_like     | Colour    | Mean saturation below 15.                      |
| glcm_contrast         | Texture   | GLCM contrast.                                 |
| glcm_dissimilarity    | Texture   | GLCM dissimilarity.                            |
| glcm_homogeneity      | Texture   | GLCM homogeneity.                              |
| glcm_energy           | Texture   | GLCM energy.                                   |
| glcm_correlation      | Texture   | GLCM correlation.                              |
| glcm_asm              | Texture   | Angular second moment.                         |
| freq_low / mid / high | Frequency | Relative FFT magnitude in radial bands.        |
| phash                 | Hash      | 64-bit perceptual DCT hash.                    |
| ahash                 | Hash      | 64-bit average hash.                           |
| md5                   | Hash      | MD5 of analysed RGB bytes.                     |
| is_corrupt            | Status    | True when file reading/stat calculation fails. |

## Appendix B — summary() Schema

```
{
  "inventory": {
    "total", "valid", "corrupt", "corrupt_paths",
    "format_distribution", "colour_mode_distribution",
    "dtype_distribution"
  },
  "spatial": {
    "height", "width", "aspect_ratio", "megapixels",
    "file_size_kb", "orientation_distribution"
  },
  "pixel_stats": {
    "dataset_mean_rgb",
    "dataset_std_rgb_across_images",
    "dataset_pixel_std_rgb",
    "per_image_mean_rgb"
  },
  "quality": {
    "brightness", "contrast", "sharpness", "noise_estimate",
    "entropy", "compression_score", "blurry_count",
    "blurry_fraction", "overexposed_frac", "underexposed_frac"
  },
  "colour": {
    "saturation", "colour_temp_distribution",
    "grayscale_like_count", "grayscale_like_fraction"
  },
  "texture": {...},
  "frequency": {...},
  "duplicates": {
    "exact_duplicate_groups", "n_exact_duplicate_groups",
    "near_duplicate_groups", "n_near_duplicate_groups"
  },
  "labels": {
    "label_distribution", "class_imbalance_ratio"
  }
}
```



## Appendix C — Metric and Threshold Reference

| Metric                | Definition / threshold                                                             |
|-----------------------|------------------------------------------------------------------------------------|
| Brightness            | $\text{mean}(0.299R + 0.587G + 0.114B)$                                            |
| Contrast              | $\text{std}(\text{luminance})$                                                     |
| Sharpness             | $\text{variance}(\text{Laplacian}(\text{grayscale}))$                              |
| Blurry flag           | $\text{sharpness} < \text{blur\_threshold}$ ; default 50                           |
| Noise estimate        | $\text{std}(\text{grayscale} - \text{GaussianBlur}(\text{grayscale}, 5 \times 5))$ |
| Overexposed           | $\text{luminance} > 245$                                                           |
| Underexposed          | $\text{luminance} < 10$                                                            |
| Grayscale-like        | $\text{mean HSV saturation} < 15$                                                  |
| Warm colour           | $\text{mean R} / \text{mean B} > 1.1$                                              |
| Cool colour           | $\text{mean R} / \text{mean B} < 0.9$                                              |
| Near duplicate        | pHash Hamming distance $\leq \text{phash\_threshold}$ ; default 10                 |
| Orientation landscape | $\text{aspect ratio} > 1.05$                                                       |
| Orientation portrait  | $\text{aspect ratio} < 0.95$                                                       |

## Appendix D — Glossary

| Term                  | Definition                                                              |
|-----------------------|-------------------------------------------------------------------------|
| aHash                 | Average hash: 8×8 grayscale comparison against the image mean.          |
| Class imbalance ratio | Largest class count divided by smallest class count.                    |
| Corrupt image         | A resolved file that OpenCV cannot read or analyse successfully.        |
| DCT                   | Discrete cosine transform used by the pHash implementation.             |
| Entropy               | Shannon entropy of the grayscale intensity histogram.                   |
| FFT                   | Fast Fourier transform used for radial frequency-band summaries.        |
| GLCM                  | Grey-Level Co-occurrence Matrix used for texture features.              |
| Laplacian variance    | Edge-response variance used as the sharpness measure.                   |
| MD5                   | Byte-level digest used for exact duplicate grouping.                    |
| pHash                 | Perceptual hash designed to remain similar for visually similar images. |
| Saturation            | HSV S channel, representing colourfulness relative to brightness.       |

## Appendix E — Acknowledgements

---

VisEDA (ImageEDA) builds on the excellent work of the following open-source projects: OpenCV (image reading and colour transformations), NumPy (array operations), Matplotlib (visualisations), scikit-image (GLCM texture analysis), scikit-learn (K-Means dominant colour extraction), and Pillow (image format support).

The cats-and-dogs dataset used throughout this book is the Microsoft Kaggle Cats and Dogs Dataset, available at <https://www.microsoft.com/en-us/download/details.aspx?id=54765>.